

Collaboration, Process Control, and Fragility in Evolutionary Product Development

Tor Erlend Fægri and Geir Kjetil Hanssen, *SINTEF Information and Communication Technology*

A software company's transition from a plan-based development process to an agile, evolutionary process reveals both the benefits and demands it places on an organization.

Evolutionary development is an agile software engineering approach that embraces continuous customer collaboration to assist the construction of a gradually evolving product. Its benefits and challenges are revealed in a longitudinal study we performed of a medium-sized company's transition from a traditional plan-based software process to an evolutionary process. The transition occurred in four consecutive product releases during a two-year period in 2004–2005. On the basis

of discussions with Tom Gilb, the company selected Evo,¹ an evolutionary project management method that Gilb first detailed in 1981.² Evo develops the final product iteratively as new requirements are discovered, constructed into the code, and subsequently tested.³ Stakeholders give feedback to each product iteration to guide the development of the next iteration. Integration test suites are performed before the final system test.

Evo is a rigorous process encompassing conscious risk management, similar to the later-developed Spiral model.⁴ However, each iteration is ultimately driven by the highest possible stakeholder value. Despite rigidity in process control, Evo builds on commonly known agile principles: continuous customer

collaboration, direct feedback to frequent releases of working software, and change responsiveness. Here, we detail the company's experiences with adopting, refining, and maturing Evo. Our aims are to share our insights regarding the organizational impacts of evolutionary development and to fill in some gaps in current knowledge about agile methods and effects.^{3,5} The "Study Method" sidebar describes how we collected and analyzed data.

Background

CompNN (a pseudonym) is a Norwegian software company that provides a packaged software product, ProdNN, for marketing and customer surveys. The company was founded in 1996 and has since grown steadily to about

80 employees with offices in Norway, Vietnam, the UK, and the US. The company shifted gradually from building custom applications to a packaged product. Today, ProdNN licenses constitute most of the company's revenue.

CompNN used a noniterative variant of the plan-based waterfall process to manage product development.⁶ When the company introduced the process, it provided the structure necessary to maintain a core product, which the shift from custom development to packaged product required. The process emphasized thorough requirements engineering up front. Analysis, specification, design, code, module test, system integration, acceptance testing, and delivery were performed sequentially and diligently, although with only modest reflection on intermediate results. Initially, the process seemed to address the company's problems by establishing control and quality management, but it eventually exposed new challenges. For example, its inflexibility incurred high costs in the face of unclear or changing requirements.

CompNN also saw a potential for improving its dialogue with customers. Packaged-product customers asked for closer collaboration with CompNN. As one customer said, "Input to the development plans was very informal. We only saw the products released by CompNN." Product planning involved mainly a collaboration between R&D and management. A CompNN representative explained: "Development was mainly driven by casual discussions between R&D and management. Gradually, the need to structure the work better and allocate responsibilities for the research ahead of product planning materialized." The somewhat ad hoc product planning failed to address many existing business requirements from end users. Another CompNN representative said, "We were really only focusing on features. Other qualities were added last, and only in very vague terms." And another representative commented, "By focusing primarily on features, we somehow forgot to think about how these features should be used and [about] the user-perceived quality of these features."

Because the first feedback to the product came late—during acceptance testing—little time was available to improve the software before delivery. Also, project effort estimates were frequently too optimistic, forcing developers to cope with intense amounts of overtime near the

Study Method

CompNN had no defined software process improvement goals prior to introducing Evo. Therefore, we used an open, qualitative, longitudinal case study to investigate potential costs, gains, and risk factors. We covered four product releases produced during 2004–2005.

Data collection

The main data source was interviews with several persons filling Evo's three most central roles: developers (six persons randomly selected), collaborating customers (one person from each of the three customers), and all project management team members (five persons). We interviewed the first group after the ProdNN version 10.0 release, using a postmortem technique. We followed an interview guide to conduct semistructured interviews with the two latter groups at various times during the study. Altogether, we conducted nine interviews, taping them all.

Additionally, we've had numerous discussions with the company's quality assurance manager and chief technology officer.

Data analysis

We transcribed the interviews for detailed textual analysis. We analyzed the results by interview group, using constant comparison.¹ The software tool NVivo supported our analysis by carefully tagging the data and grouping analytical fragments into larger constructs.

Reference

1. C.B. Seaman, "Qualitative Methods in Empirical Studies in Software Engineering," *IEEE Trans. Software Eng.*, vol. 25, no. 4, 1999, pp. 557–572.

release date. In the pressure to deliver functionality, testing would often give way to development. As a result, the delivered product had many errors. Furthermore, company management saw a trend of declining R&D motivation. They feared that these problems would limit the product's competitiveness.

In November 2003, CompNN's quality assurance (QA) manager and the chief technology officer (CTO) met Tom Gilb at a software engineering research forum, where he gave a presentation on Evo. The company representatives found the ideas appealing, and within a month CompNN decided to apply some key Evo practices to the next product version, due for release in May 2004.

Evo practices at CompNN

Several engineering domains have used the Evo project management method, and Gilb discusses its applicability to systems engineering in his latest book.¹ Evo includes many practices, but here we discuss only those adopted and shaped by CompNN.

	A	B	C	D	E	F	G	BX	BY	BZ	CA
1											
2		Current Status	Improvements	Goals			Step9				
3							Recoding				
4							Estimated impact		Actual impact		
5		Units	Units	%	Past	Tolerable	Goal	Units	%	Units	%
6					Usability.Replacability (feature count)						
7		1,00	1,0	50,0		2	1	0			
8					Usability.Speed.NewFeaturesImpact (%)						
9		5,00	5,0	100,0		0	15	5			
10		10,00	10,0	200,0		0	15	5			
11		0,00	0,0	0,0		0	30	10			
12					Usability.Intuitiveness (%)						
13		0,00	0,0	0,0		0	60	80			
14					Usability.Productivity (minutes)						
15		20,00	45,0	112,5		65	35	25	20,00	50,00	38,00
20					Development resources						
21			101,0	91,8		0		110	4,00	3,64	4,00

Figure 1. Impact estimation table from ProdNN 8.5. Current goals are listed vertically, and their impacts in planned iterations appear horizontally.

Requirements engineering and project management

Arguably, Evo's most important attribute is project management guided by impact-based effort prioritization and formalized requirements engineering. In Evo, *impact* quantifies the software qualities that give customers the greatest value, and *goals* denote measurable requirements. In Evo jargon, a goal isn't simply a statement of functionality, such as "The system should ...," but a statement of the performance the software system must or should have. This approach explicitly helps distinguish requirements from designs.¹

To keep track of the goals and monitor progress, each project uses *impact estimation tables*. An IET is a spreadsheet that lists the project's planned iterations horizontally and the current goals vertically. The goals are supplemented by a *tolerable* metric, which is a numeric "worst-acceptable case" constraint level for the quality requirement and helps make the goal's sensitivity explicit. For example, in figure 1, Usability.Productivity is a goal measured as "the time in minutes for the system to generate a survey." The goal is 25 minutes, but the tolerable metric is 35 minutes. In Evo, *design ideas* are proposed solutions to reach the goals. In the figure, the design idea for iteration 9 is "Recoding."

CompNN produces two product releases each year. Each release is broken down into two to five parallel time-boxed projects with an R&D team of two to five developers each, including one appointed project manager (PM). Each project focuses on specific

ProdNN modules or types of functionality. The product management team (PMT), composed of five senior employees, is responsible for creating and maintaining product development plans in close cooperation with customers. Input from the CTO, executive management, and customers (for example, from yearly surveys) is important in this work.

Product development plans can span multiple releases, and the PMT together with the CTO are responsible for ensuring that the IETs conform to them. Before release initiation, selected customers are invited to supplement the plans with additional requirements. During the initiation phase, which spans roughly two weeks, the PMT together with the respective PMs capture the most promising design ideas to address the initial goals and enters them in the IET for future iterations. As CompNN's CTO explained, "This plan helps us ensure adherence to an overall technical strategy for product development."

Process scheduling

Evo is a highly scheduled process. Figure 2 presents the *Evo week*, a schedule that CompNN iterates each week to define tasks for the project's defined roles. Most planning and coordination work occurs on Friday, preparing the development team for a quick start on Monday morning. In each iteration, the project teams implement the planned design idea, make a new version of the software system available to selected stakeholders, and compare the effort's results to the goal. In weekly project leaders' meetings, QA collaborates with the PMs

	Friday	Monday	Tuesday	Wednesday	Thursday
R&D team	PM: Send version N detail plan to CTO + prior to project mgmt meeting	R&D team: Develop test code and code for version N	R&D team: Develop test code for version N	R&D team: Develop test code and code for version N	R&D team: Complete test code and code for version N
	PM: Attend project mgmt meeting: 12:00-15:00		R&D team and PM: Meet with stakeholders to discuss action taken		
	R&D team: Focus on general maintenance work, documentation				
Stakeholder		• Use version N-1	• Meet R&D team to give feedback and discuss action taken from previous actions		
CTO	• Approve/reject design & step N		• System architect to review code and test code		
	• Attend project mgmt meeting: 12:00-15:00				
QA	• Run final build and create setup for version N-1	• Follow up CI	• Review test plans and tests	• Review test plans and tests	• Review test plans and tests
	• Install setup on test servers (external and internal)	• Review test plans, tests	• Follow up CI	• Follow up CI	• Follow up CI
	• Perform initial crash test and then release version N-1				

Figure 2. The Evo week. Planning and coordination of work, according to project roles, occurs on Friday to jumpstart work on Monday morning.

and CTO to review the stakeholders' feedback and decide whether the work meets the requirements (to either the tolerable or the goal level). When the work meets specified goal levels, the IET is updated with new goal and tolerable requirement levels, according to stakeholders' feedback. This concludes an iteration.

At the start of a new iteration, the PM reviews each requirement according to how the iteration's design idea impacts it. The IET simplifies the consideration of a design idea's side effects on the range of current goals. Occasionally, the project team might suggest revisions to design ideas at this point. IETs function both as the overall development plan and a progress record. Using metrics for each goal ensures that any person within the company can easily determine the development's status. The QA manager and CTO jointly fill the role of manager for the release.

CompNN established a technical framework for *continuous integration* to support more frequent builds and tests. CI integrates a toolset that almost fully automates the software build process and the execution of partial tests of the software several times a day. At present, CI includes Starteam (from Borland) for source control, NUnit for unit testing (open source), FxCop for code analysis (open source), NAnt for build management (open source), and CruiseControl.Net for coordina-

tion and reporting (open source). CI is configured to automatically upload the product to test servers each Friday.

Adhering to the Evo week requires disciplined teamwork. The PMT appoints a stakeholder to represent important project goals. A three-person QA group is responsible for deploying testable product versions to the stakeholders. The QA group also ensures follow-up on any reported discrepancies from CI. Its activities run in parallel with the unit testing performed by R&D. QA is responsible for reviewing the test plans and results from the tests run by R&D. Thus, QA's role is to ensure that all issues detected during unit testing are eventually resolved.

The transition experiences

The Evo implementation for each of the four product releases involved a different configuration and yielded different lessons.

The Evo pilot: ProdNN release 8.5

CompNN established the PMT a few weeks before initiating the 8.5 release of ProdNN. Collaborating with customers, PMT created a list of important improvements. The 8.5 release plan was divided into four parallel projects focusing on distinct product modules. This resulted in small, easily manageable R&D teams. Each project followed the Evo week,

**For R&D,
the change
to Evo
significantly
improved
the comfort
level and
motivation.**

employed IETs, and used the CI framework. Additionally, each project included one PMT member who gave feedback to the iterations, simulating customers.

Although the change to Evo might appear dramatic, many interviewees commented that they had expected a change from management for some time (“something had to happen”). A PMT member went so far as to say, “Whatever changes we implemented, it would probably be a good thing.” Because CompNN was experiencing a difficult period, the organization was generally enthusiastic about the pilot project, which got off to a flying start.

CompNN observed many positive effects as it adopted the new, highly structured development approach. One benefit, applying primarily to R&D, was a reduction in the intense overtime work at a project’s end. Reviewing suggested improvements iteratively resulted occasionally in postponements to a later release. For R&D, the change to Evo significantly improved the comfort level and motivation. Through nightly builds, supported by the CI framework, the company always had a running version ready for testing and feedback, leaving R&D more time and quiet to develop code.

Maintaining CI, a premise for Evo collaboration and control, required substantially more effort than anticipated. This related primarily to maintaining the tests. Furthermore, establishing quality metrics in cooperation with PMT succeeded only partially. We learned that some developers continued their old “feature counting” practice. We weren’t overly surprised by these findings—establishing good requirements and associated metrics is inherently hard.⁷ For the goals that did include quality metrics and effective measurements, developers appreciated the added certainty. They expressed this in the context of “interest in what we do on a weekly basis—not in a four-monthly basis as before.”

The general experience with the Evo pilot was positive. It ran for five months, required only modest preparations, and gave the CTO, R&D, QA, and PMT confidence with the new process.

Inviting the customers: ProdNN 9.0

CompNN decided to expand the use of Evo practices in the following 9.0 release, commencing May 2004. PMT carefully selected three customers to participate as collaborating

stakeholders on the basis of three criteria: economic value, experience with the product, and complementary interests in ProdNN. The customers received no compensation for participation, so PMT felt certain that their collaboration was based on interest and willingness to shape the product. One of the customers said, “Since 2001 we had discussed the possibility of closer collaboration with development. When CompNN turned to Evo, it became a reality.” This customer reported having spent two person-weeks on preparing product plan input and two person-weeks on evaluation and feedback.

The three customers were assigned to individual projects. PMT’s role changed correspondingly—from surrogate customers to facilitators of the collaboration between the customers and R&D. Stakeholders gave feedback in project meetings via telephone conferencing. Because the project teams were located in Norway and two stakeholders were located abroad (US and Canada), establishing the technical infrastructure for software sharing posed a technical challenge. CompNN decided to make the software versions available for the stakeholders through an application service provider (ASP). CompNN supplemented the meetings with screen sharing and occasional technically focused meetings to discuss complex issues. R&D soon accepted the customer dialogue as the most significant mechanism for refining goals and doing functional testing. This interaction generated valuable knowledge used to update the IETs and subsequently to dynamically control the R&D efforts.

Collaborating directly with the customers increased the developers’ confidence compared to collaborating with PMT. One developer said, “It’s the feedback that works much better. Now we get feedback very fast.” Another PMT member commented on the explicit focus on software qualities: “... because we now focus on qualities as opposed to features we now are capable of giving the customers what they need, not necessarily what they want.” The tighter relationship with customers enabled CompNN to better understand the underlying business problems. A developer stated: “It was a very satisfying situation because we managed to define measurable goals that we knew were important to the stakeholder.” The collaboration provided a fruitful environment in which valuable knowledge was externalized and

transformed into the evolving product. A PMT representative with additional documentation and training-materials responsibilities commented: “Our job got easier because we knew what was coming at an early point in the release. For the first time, we were finished one week before the release date.” As a result, the sensation of being in control improved the comfort for both R&D and PMT and further increased their motivation.

The rapid iterations in Evo made some requested quality goals unrealistic. One customer had business requirements related to the performance of certain administrative tasks in the product, such as installation of new databases and other system configuration duties. The project couldn’t include these goals for practical reasons. Giving feedback on such goals would have required the customer to test on a local installation as well as longer iterations.

As the release progressed, cracks started to appear. The weekly schedule became too fast-paced and R&D complained that too little time was left for testing, resulting in increased errors and discomfort in R&D. This exposed a *fragility* in the process, primarily its dependence on timely deliveries and effective collaboration. We define fragility as the level of a process’s sophistication as given by the number of preconditions and dependencies that must be met. This problem was intolerable, and roughly midway through the release, CompNN decided to try a suggestion from Gilb—namely, the *green week*. Rather than spending each Friday for corrective maintenance, R&D now dedicated a full week to this task. The project could add no new functionality during this green week. The PMs adjusted schedules so that stakeholders didn’t do anything at this time.

The experiences from introducing the green week were mixed. First, it helped address the problem of too little time for testing. The interviewed developers agreed that the green week enabled them to catch up with the testing and increased their confidence in the code. However, they also felt the cost was too high. We were puzzled and asked why. One developer said reluctantly, “Our testing tool (FX-Cop) is very cumbersome. It is not integrated with our other development tools. So the process is slow and appears as a diversion.” This illustrates the importance of efficient tools. We believe that the near-perfect vision

of a fully integrated development environment, created by CI, was severely broken by the testing tool and that this gave R&D a very negative attitude towards testing. This, in turn, explains the increasing number of errors.

Picking up some broken pieces: ProdNN 9.1

Release 9.0—and Evo—was at this point considered a success with respect to customer collaboration. However, the extreme focus on responding rapidly to feedback led to many new code errors. The green week helped, but not enough. Many of the reported problems weren’t malfunctioning features but poorly performing features that hadn’t been tested properly because of lack of time. In an effort to counter this problem, CompNN decided to run an interim release: “The objective was to enhance performance, handle change requests, reduce number of bugs, document code and improve the tests.” The release was divided into four parallel projects. PMs were told to follow the Evo practices, but only one project, called “system configuration,” was allocated an external stakeholder. The other three projects simulated stakeholders in the project group meetings with respect to the interim release’s quality goals.

The system configuration project was completed successfully with disciplined adherence to the Evo-week and a fruitful collaboration with the appointed stakeholder (CompNN’s ASP). Results for the other projects were mixed, primarily because of PMT and other management’s lack of engagement. This, in turn, created a sense of vacuum among the developers. A developer stated: “Driving the process alone is very demanding. You sit there wondering what is happening.” Confidence suffered accordingly. What they so easily had become accustomed to during the two previous releases was now taken away. Consequently, the Evo process appeared as a burden. They missed valuable feedback and interaction with a clearly defined stakeholder. Some developers also reported that motivation suffered in this release. The PMs didn’t explicitly decide to drop the Evo measurement and feedback practices—it “just happened” because following Evo in detail seemed irrelevant with no external stakeholders. However, without the direct, instructive collaboration with stakeholders, developers eventually moved away from the practice of performing weekly



**Evo's
dependency
on frequent
feedback
and fluent
decision making
contributes
to its fragility.**

measurements, and these projects partly fell back on past practices.

Experiencing the fragility: ProdNN 10.0

In release 10.0, CompNN's management again set a stronger focus on employing the Evo principles throughout all projects. The commitment to recreate the spirit of the 9.0 release was strong. The 10.0 release was divided into three projects, each to be run according to Evo principles and to include the green week. Again, it started well. R&D quickly regained their motivation through stimulating collaborations with customers. PMT also did a good job in ensuring active stakeholder engagement. Occasionally, some customers couldn't participate in the feedback meetings. For the most part, PMT was prepared for this event and had gathered enough customer input beforehand to give the developers good information. Thus, PMT functioned as the buffer for unpredictable customer availability.

But problems started to arise during the summer holidays, roughly halfway through the development period. One customer withdrew from its allocated project because the goals had been reached successfully. Other customers left and reappeared arbitrarily, causing an unpredictable supply of feedback to steer the projects. The company took little action to correct the situation, and the summer holidays also reduced the availability of key people. Lacking steering and able decision makers, the 10.0 projects experienced growing uncertainty and mistrust.⁵

Evo's dependency on frequent feedback and fluent decision making contributes to its fragility. Release 10.0 exposed this fragility. Without management's attention to keep the process components going, Evo showed signs of collapse. We weren't surprised to find a combination of reasons behind this. First, Evo's key mentors in CompNN had become overloaded with other tasks and thus failed to notice the process's gradual decay. Additionally, as this problem became evident, management failed to take corrective action. Instead of protesting loudly, R&D quietly took this responsibility themselves and ended up as process owners. Lacking other directions, they focused on adding functionality that they believed would be useful to customers. Paradoxically, the 10.0 release impressed customers with the introduction of a large number of fea-

tures. But R&D failed to understand their own limitations. They were too ambitious. The result was an inevitable all-out effort as the release date approached, pushing many of the R&D team members to exhaustion.

Discussion

Evolutionary software development promotes a collaborative environment where supplier and consumer can generate the knowledge to gradually create the optimal product. This knowledge also supports control of the software development process. CompNN reaped such benefits from adopting Evo. In addition, participants reported increasing confidence and comfort with the development process.

Companies can only exist in relationships with customers. Evo has made these relationships more collaborative for CompNN. Furthermore, constantly focusing on delivering customer value increases the synergetic value of these relationships. This becomes a significant competitive advantage, enabling more efficient innovation transfer between CompNN and the collaborating customers. Ideally, all collaborating customers should represent strategically, technically, and economically optimal product requirements, but this is—of course—impossible to achieve. The cost of including an unresponsive stakeholder is moderate in a responsive organization, which can acknowledge the problem in a few iterations and then begin steps to correct it. Furthermore, during the time it takes to replace the stakeholder, vigilant PMT and project teams can still reap benefits for the organization. In the friendly, collaborative environment fostered by Evo, CompNN extracted useful knowledge about how customers used the product. The company also used this time to try out new ideas with the customers.

And here lies a key lesson from this study: maintaining a watchful, vigorous stakeholder management capability is paramount for Evo's success.

Evo increased the product's exposure and resource usage compared to the original process. One PMT member, working abroad, noted that "Evo makes ProdNN and the development process more visible throughout the company and also among our key stakeholders." More people can observe or criticize prioritizations. This can help maintain focus

on generating value, but it also influences the nature of the demands on the product. A CompNN representative said, “I think we have improved our ability to think in terms of the real business problem of the stakeholders.” This follows implicitly from the increase in direct stakeholder feedback. However, R&D saw “a lack of attention to software engineering principles.” Finding the balance in this dynamic environment is a well-known challenge.⁵ Therefore, companies should select stakeholders with a particular interest in the product as an engineering artifact (called “internal stakeholders” in Evo) at intervals to counter architectural erosion and subsequent excessive costs of implementing improved quality.

We also learned that short cycles of specification, design, development, and testing reduce tolerance toward inefficient tools. Potential adopters should keep this in mind.

The continuous stream of knowledge from the collaboration with stakeholders is a main factor guiding the development process. We believe these rapid, direct feedback loops explain most of the increase in employee motivation and enthusiasm, particularly in R&D. In this respect, the IETs functioned as lightweight yet effective decision-making tools. Rapid feedback on developed code, appropriate metrics, and effective decision making help reduce uncertainty and thus increase developer motivation. CompNN has embraced user involvement, assuming it’s a key source of innovation.⁸

The positive experiences reported by stakeholders stemmed mainly from a feeling of being listened to and able to affect the development.

Evo is a sophisticated process, highly dependent on diligent fulfillment of the specified roles and activities. This sophistication is also Evo’s Achilles’ heel. Evo is very vulnerable to irregularities. This process fragility increases with the frequency of iterations, which put higher demands on the timeliness of role fulfillment and activities. Unresponsive stakeholders expose this fragility. Our observations from the ProdNN 10.0 release demonstrate this. CompNN’s experiences with weekly iterations showed this schedule to be too demanding. An adopting organization must take multiple factors into account when determining the frequency. Evo strongly recommends iterations consuming less than 5 percent of the total project funding.¹ CompNN’s projects used between 10 and 20 iterations, each con-

About the Authors



Tor Erlend Fægri is a researcher on software process improvement and software architecture at SINTEF ICT, a research division of Norway’s largest independent research institution. He has an MSc in computing science from the University of Glasgow. Contact him at SINTEF ICT, No-7465 Trondheim, Norway; tor.e.fegri@sintef.no.

Geir Kjetil Hanssen is a researcher on software process improvement and methodologies at SINTEF ICT, a research division of Norway’s largest independent research institution. He has an MSc in informatics from the University of Trondheim. Contact him at SINTEF ICT, No-7465 Trondheim, Norway; geir.k.hanssen@sintef.no.



suming 5–10 percent of the total project funding. Even this is too frequent, as the experience from the 9.0 and 10.0 releases showed. CompNN plans to experiment with iterations spanning two weeks instead of one in the coming release, hoping to contain fragility and leave more time for development by reducing the coordination overhead.

Potential adopters of this process must also consider their available resources for managing stakeholder involvement when determining the iteration frequency. Additionally, if the risk of missing stakeholders’ real concerns is considered low or moderate, companies might want to start with longer iterations and then shorten them to train the organization for Evo. If the risk is considered high, they can use more frequent iterations but then with a strong attention to effective stakeholder management. Success is important not least because the consequences of abandoning this highly motivating way of working can be devastating.

Evo certainly isn’t a process for the faint of heart. As we’ve seen, implementing and maintaining the organizational and technical capability to support weekly iterations is expensive and requires a high degree of discipline at many organizational levels. Roles other than those we’ve studied might have been affected, thus constituting additional costs to adopting Evo. Organizational changes take time and effort. During the two years of our study, CompNN implemented and matured a new development

process. However, a company should consciously and regularly consider whether it's necessary to adapt their development processes. CompNN has done this by introducing the green week in the 9.0 release and by extending the iterations to two weeks in the coming release. Other adaptations are likely in future releases.

As Frederick Brooks so eloquently put it, there are no silver bullets in software engineering.⁷ Nevertheless, improvement through evolution is undeniably a powerful practice. ☞

Acknowledgments

The Research Council of Norway sponsored this study through the SPIKE research program under grant 156701/220. We thank CompNN for their kind cooperation.

References

1. T. Gilb, *Competitive Engineering: A Handbook for Sys-*

tems Engineering, Requirements Engineering, and Software Engineering using Planguage, Elsevier Butterworth-Heinemann, 2005.

2. T. Gilb, "Evolutionary Development," *ACM Software Eng. Notes*, Apr. 1981, p. 17.
3. P. Abrahamsson et al., "Agile Software Development Methods—Review and Analysis," *VTT Publications*, no. 478, VTT Electronics, 2002.
4. B.W. Boehm, "A Spiral Model of Software Development and Enhancement," *Computer*, vol. 21, no. 5, 1988, pp. 61–72.
5. B. Boehm, "Get Ready for Agile Methods, with Care," *Computer*, vol. 35, no. 1, 2002, pp. 64–69.
6. W.W. Royce, "Managing the Development of Large Software Systems," *Proc. IEEE Westcon*, 1970; reprinted in *Proc. 9th Int'l Conf. Software Eng. (ICSE 87)*, IEEE Press, 1987, pp. 328–338.
7. F.P. Brooks, "No Silver Bullet: Essence and Accidents of Software Engineering," *Computer*, vol. 20, no. 4, 1987, pp. 10–19.
8. E. Carmel and S. Becker, "A Process Model for Packaged Software Development," *IEEE Trans. Eng. Management*, vol. 42, no. 1, 1995, pp. 50–61.

For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.

CALL FOR ARTICLES

Security for the Rest of Us: An Industry Perspective on the Challenge of Secure Software

The public need for good software security becomes more acute every day. Typical activities such as purchasing products and services, conducting business, and holding elections increasingly depend on secure software. While security was once a specialty of interest to only a small number of developers, it's now critical for almost all software developers, project managers, and decision makers. The worldwide software industry includes thousands of vendors, from massive enterprises to one-person shops, and the industry as a whole has to face the software security challenge.

PUBLICATION: January/February 2008

SUBMISSION DEADLINE: 1 July 2007

TOPICS OF INTEREST:

- Practical tools and methods for detecting or preventing security-relevant defects
- Practical approaches to incorporating security as part of the software development process
- Attacks and vulnerabilities—common ways that security fails in modern industrial software
- The economic motivation for creating secure software

GUEST EDITORS:

- **Konstantin Beznosov**, University of British Columbia; beznosov@ece.ubc.ca
- **Brian Chess**, Fortify Software; brian@fortifysoftware.com

IEEE
Software

WWW.COMPUTER.ORG/SOFTWARE

For **author guidelines** and **submission details**, contact the magazine assistant at software@computer.org or go to www.computer.org/software/author.htm. Specify that you are submitting for the "Security for the Rest of Us" special issue. A **detailed call** is at www.computer.org/software/cfp.htm.